# Backend - 4. Docker Multi Container

Pada sesi ini kita akan belajar cara menggunakan **Dockerfile** untuk membuat image berdasarkan basis image tertentu, kemudian menggunakan **docker-compose** untuk menjalankan dan melakukan konfigurasi terhadap beberapa container sekaligus.

## Dockerfile

**Dockerfile** adalah sebuah file yang berisi instruksi dan perintah yang digunakan oleh **Docker Engine** untuk membangun sebuah **Image Docker**. Dockerfile berisi perintah dan langkah pull dan modifikasi image dengan menambahkan kode, paket, runtime, lingkungan, dan pustaka yang dikemas dalam satu kesatuan.

Beberapa perintah yang umum digunakan dalam Dockerfile meliputi:

1. **FROM**: menentukan **image basis** yang akan digunakan untuk proses build.
2. **RUN**: menjalankan perintah di dalam image yang sedang dibuat.
3. **COPY**: menyalin file dari sistem host ke dalam lingkungan Docker image yang sedang dibangun.
4. **ADD**: Sama seperti COPY, tetapi ADD juga dapat mengekstrak arsip (ZIP, TAR) secara otomatis.
5. **WORKDIR**: mengatur direktori kerja saat menjalankan perintah dalam image.
6. **EXPOSE**: menentukan port yang akan dibuka oleh image Docker ketika dijalankan.
7. **CMD**: menentukan perintah default yang akan dijalankan ketika container dijalankan.
8. **ENTRYPOINT**: menentukan perintah utama yang akan dijalankan saat container dijalankan.
9. **ENV**: untuk mengatur variabel lingkungan.

### Contoh Studi Kasus: Dockerfile untuk Apache + PHP Web Server:

Pada studi kasus ini kita akan membuat image PHP Web Server berbasis Apache dengan modifikasi penambahan extension yang diperlukan untuk koneksi MySQL. Langkahnya adalah sebagai berikut:

- Buat sebuah direktori proyek baru dan buka dengan VS-Code
- Buat file dengan nama `Dockerfile` dan isi dengan konten sebagai berikut:

```
FROM php:8.2-apache

WORKDIR /var/www/html

RUN apt-get update && \
    apt-get install -y libpng-dev && \
    docker-php-ext-install pdo pdo_mysql gd

RUN docker-php-ext-install mysqli && docker-php-ext-enable mysqli

EXPOSE 80

CMD ["apache2-foreground"]
```

- Buka built in terminal VS-Code untuk menjalankan perintah build image:

```
docker build -t {username}/{nama_image} .
```

- Run image tersebut:

```
docker run -v ./www:/var/www/html -p 8081:80 {username}/{nama_image}
```

# Docker Network

Docker menyediakan kemampuan untuk mengelola jaringan antar container untuk mendefinisikan bagaimana **container** berkomunikasi satu sama lain dan dengan jaringan luar.

1. **Jenis Network di Docker:**
   - **Bridge**: Network default untuk container. Jika network tidak ditentukan, container akan terhubung ke bridge network.
   - **Host**: Menghapus isolasi jaringan antara container dan host Docker.
   - **Overlay**: Memungkinkan container yang ada di berbagai host Docker untuk berkomunikasi.
   - **Macvlan**: Memberi container alamat MAC sendiri, membuatnya tampak seperti perangkat fisik di jaringan.
2. **Membuat dan Menggunakan Network:**
   - Buat network dengan perintah docker network `create --driver bridge mynetwork`.
   - Selanjutnya kita bisa menjalankan container dengan mapping ke network yang terhubung menggunakan perintah `docker run --network mynetwork myimage`.
3. **Komunikasi antar Container:**
   - Container pada network yang sama dapat berkomunikasi satu sama lain menggunakan nama container sebagai hostname.
   - Untuk aplikasi multi-container, ini memudahkan konfigurasi dan manajemen layanan yang bergantung satu sama lain.

# Docker Compose

**Docker Compose** adalah alat yang memungkinkan kita menjalankan **multi-container application environments** berdasarkan definisi yang diatur dalam file **YAML** (biasanya bernama `docker-compose.yml`). Beberapa kegunaan docker compose adalah:

1. **Service**: Mendefinisikan setiap layanan dalam aplikasi yang dijalankan, seperti server web, basis data, atau layanan aplikasi.
2. **Image**: Menentukan image Docker yang akan digunakan untuk mengatur setiap **service**.
3. **Volume**: Memungkinkan penyimpanan data di luar kontainer dan mempertahankan datanya bahkan setelah kontainer berhenti atau dihapus.
4. **Port Mapping**: Menentukan port yang akan dihubungkan antara host dan kontainer.
5. **Environment Variables**: Mengatur variabel lingkungan yang dibutuhkan oleh aplikasi pada container-container yang terlibat.
6. **Network Configuration**: Mengatur konfigurasi jaringan antara kontainer.

## Contoh Studi Kasus: Wordpress dengan Database MySQL & Redis

Kali ini, kita akan melakukan deployment Wordpress dengan docker menggunakan 3 server yaitu:

- Apache + PHP Parser: untuk menyimpan source code wordpress
- MySQL: server basis data utama untuk menyimpan data wordpress
- Redis: server basis data tambahan untuk menyimpan data cache

Langkah *deployment*-nya adalah sebagai berikut:

- Buat direktori baru pada editor (VS-Code) dan jadikan direktori tersebut direktori utama proyek
- Buat file dengan nama `Dockerfile` dan isikan koding sebagai berikut:

```
FROM php:8.2-apache

WORKDIR /var/www/html

RUN apt-get update && \
    apt-get install -y libpng-dev && \
    docker-php-ext-install pdo pdo_mysql gd \

RUN docker-php-ext-install mysqli && docker-php-ext-enable mysqli
RUN pecl install redis docker-php-ext-enable redis

EXPOSE 80

CMD ["apache2-foreground"]
```

- Selanjutnya buat juga file `docker-compose.yml` dengan isi seperti di bawah ini:

```
version: '3'
services:
  php:
    container_name: php-server
    build:
      context: .
      dockerfile: Dockerfile
    volumes:
      - ./src:/var/www/html
    ports:
      - 8080:80
    depends_on:
      - mysql
  mysql:
    image: mysql:latest
    container_name: mysql-server
    environment:
      MYSQL_ROOT_PASSWORD: sandirahasia
      MYSQL_DATABASE: wordpress_dp
      MYSQL_USER: wordpress_user
      MYSQL_PASSWORD: sandi_wordpress
    volumes:
```

```
      - ./mysql_data:/var/lib/mysql
  redis:
        image: redis:latest
        container_name: redis-server

volumes:
  mysql_data:
```

- Untuk memastikan PHP parser sudah berjalan dengan baik dan extension yang dibutuhkan untuk wordpress juga sudah terinstall dengan baik, buatlah file `info.php` pada direktori `src` dan isikan dengan kode berikut:

```
<?php
phpinfo();
```

- Buka file tersebut pada browser dengan alamat `http://localhost:8080/info.php` dan spesifikasi PHP yang sudah terinstall, pastikan bahwa extension `mysqli` dan `redis` sudah tercantum.
- Download source code wordpress terbaru dari laman resminya
- Kemudian ekstrak isinya pada direktori `src`
- Jalankan instalasi wordpress melalui browser menggunakan konfigurasi lingkungan yang sudah didefinisikan pada file **docker compose**.
- Setelah wordpress berhasil berjalan dengan baik, masuk ke halaman administrator (`http://localhost:8080/wp-admin`) dan buka menu Plugin.
- Install plugin **Redis Object Cache** dan aktifkan (jika diperlukan cek tutorial instalasi pada halaman dokumentasi plugin tersebut).
- Tambahkan konfigurasi berikut di bagian atas pada file `wp-config.php`

```
define( 'WP_REDIS_HOST', 'redis-server' );
define( 'WP_REDIS_PORT', 6379 );
define( 'WP_REDIS_PREFIX', 'dolanan' );
define( 'WP_REDIS_DATABASE', 0 ); // 0-15
define( 'WP_REDIS_TIMEOUT', 1 );
define( 'WP_REDIS_READ_TIMEOUT', 1 );
```

- Buka menu **Settings > Redis** pada halaman administrasi wordpress, kemudian klik pada tombol **Enable Object Cache**

Setelah menyelesaikan tutorial studi kasus ini, kita sudah berhasil melakukan deployment sebuah website baru menggunakan wordpress dengan basis data MySQL dan Redis sebagai object cache-nya.